순천향대학교

ARTIFICIAL INTELLIGENCE CONVERGENCE SOFTWARE
순천향대학교 인공지능 융합 소프트웨어 연구실

# Unit Step Function, Sigmoid, BCE, Logistic Regression

박성진

발표 | 박성진 010 – 9450 – 6778 ppssjj020222@gmail.com 순천향대학교 멀티미디어관

# CONTENTS

01 Unit Step Fuction

02 Sigmoid

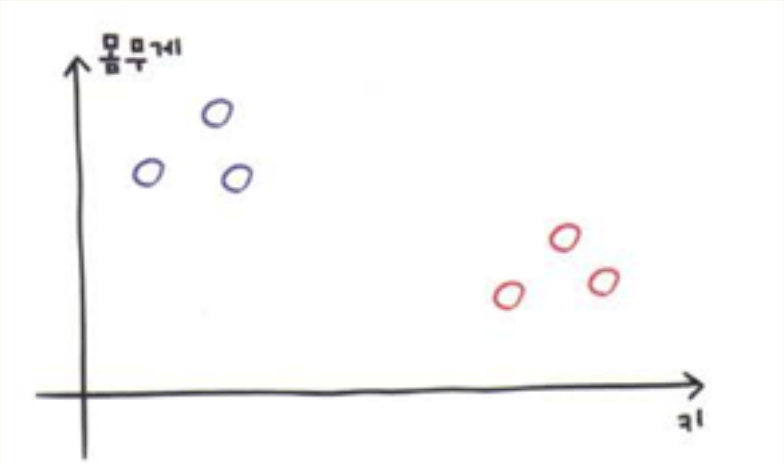
03 BCE

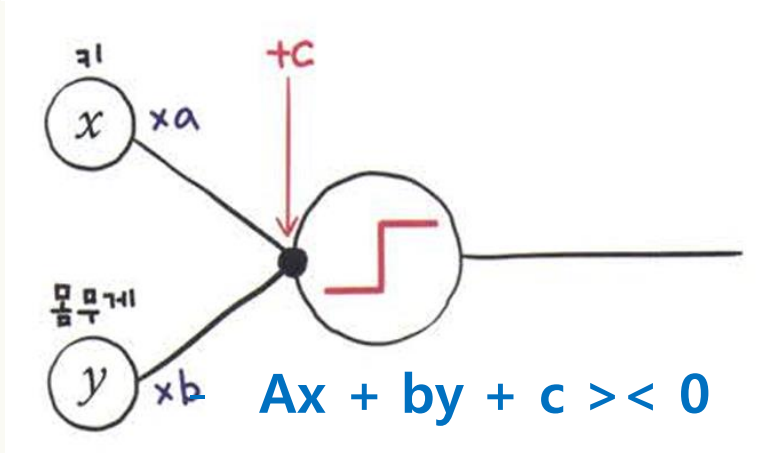
04 Logistic Regression

# 01 | UnitStepFuction 이진 분류

## Non-linear activation 으로 이진 분류 문제를 풀어보자

- 입력 : 키와 몸무게 & 출력: 1 (빼야 할 사람) 또는 0 (쪄야 할
- Step 1. 데이터 모으기(분류 경계 데이터가 많을 수록 Good)
- Step 2. 모델 만들기
  - Unit step function 이용해서 분류해보자! (입력 몇 개? 몇 개?)
  - Hidden layer 없이 unit step funtion을 activation으로 면 퍼셉트론이라 함



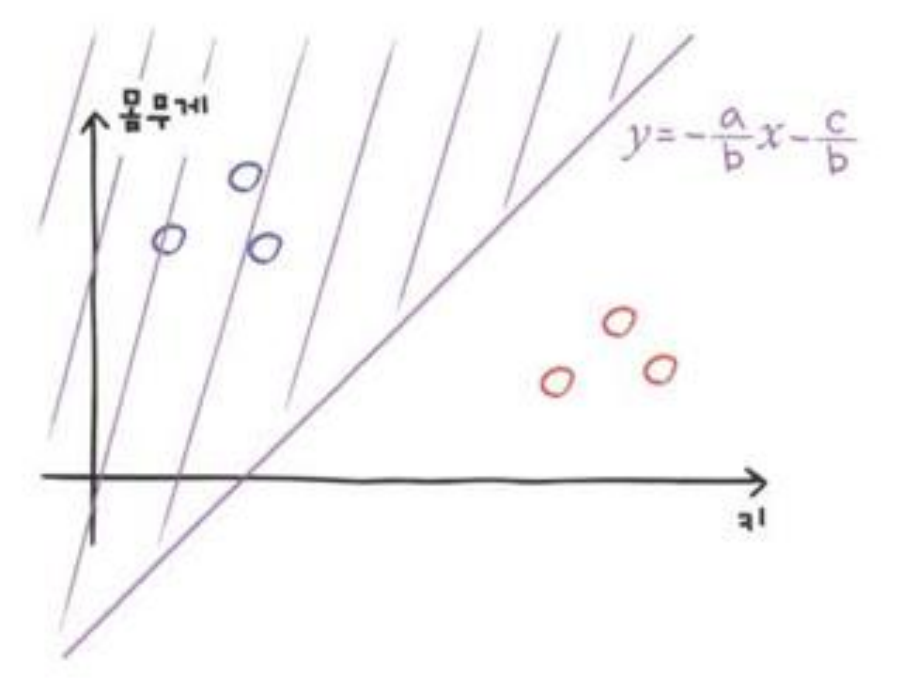



# 01 | UnitStepFuction 이진 분류

## Non-linear activation 으로 이진 분류 문제를 풀어보자

- **Step 3. 모델 학습시키기**
  - 분류 경계가 선형이 된다! 경계가 선형이면 선형 분류!

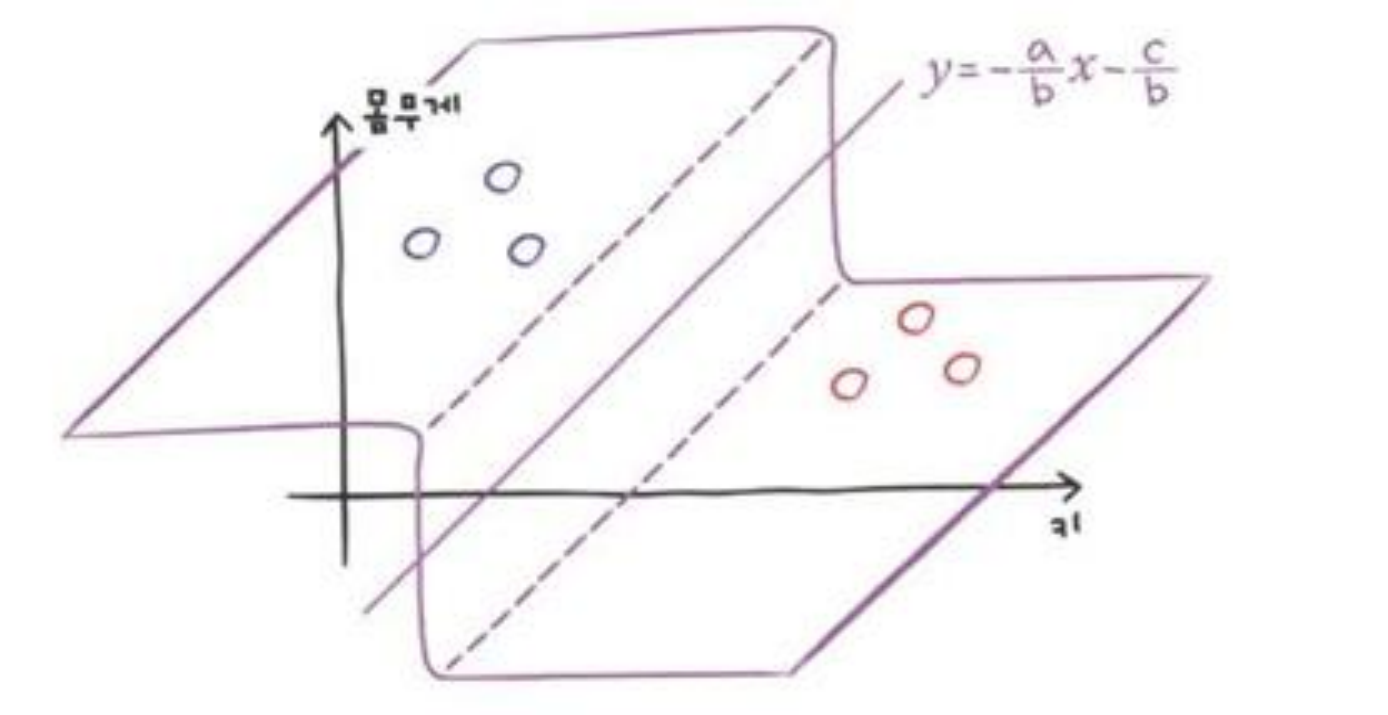
  - Ax + by + c > 0
  - By > -ax – c
  - Y > -(a/b)x - (c/b)
  - 분류 경계선 Y = -(a/b)x - (c/b)



# 01 | UnitStepFuction 이진 분류

## Non-linear activation 으로 이진 분류 문제를 풀어보자

- **Step 4. 모델 테스트하기 (새로운 데이터 예측)**
  - 3D로 표현되는 것 확인 (입력과 출력과의 관계는 비선형)



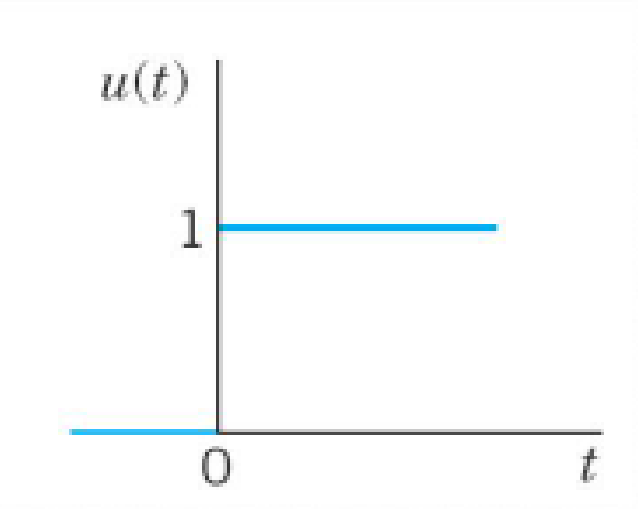

- **입력과 출력 간의 관계는 비선형적**
- **분류 경계선은 선형**

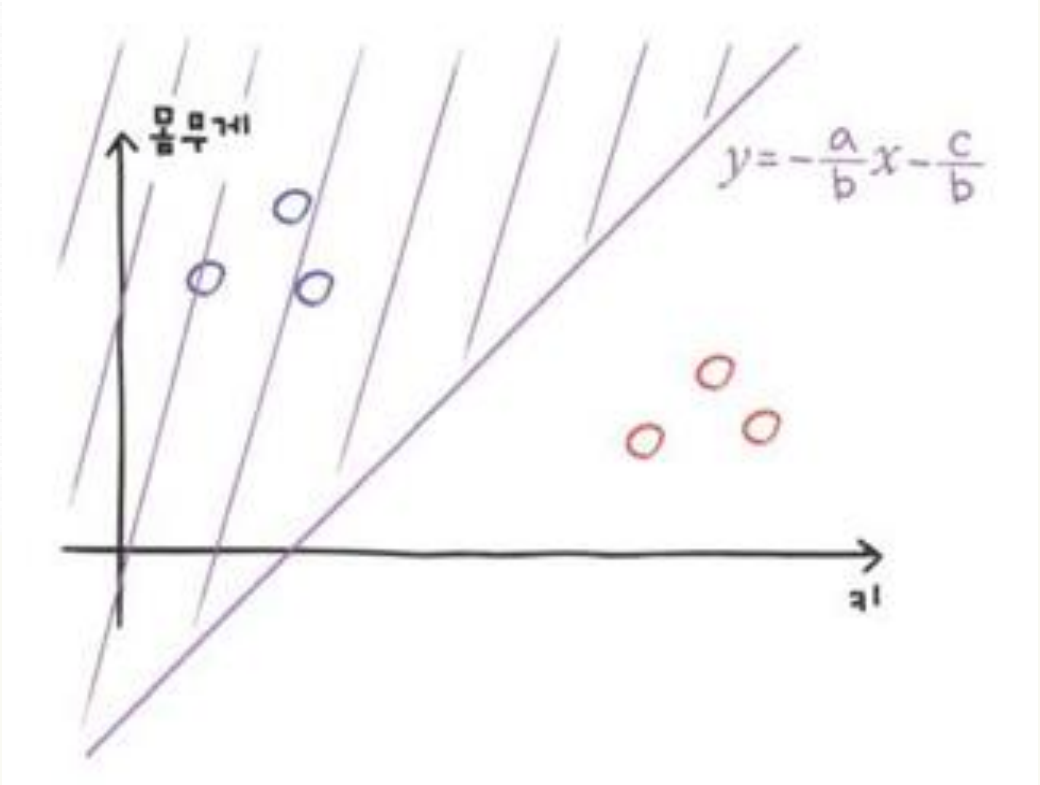
# 01 | UnitStepFuction 한계

## Unit Step Fuction의 문제점

- 미분 불가능 : Unit Step Fuction은 0에서 미분 불가, 모든 지점에서 미분값이 0

- 극단적 분류 : 출력값이 0 or 1



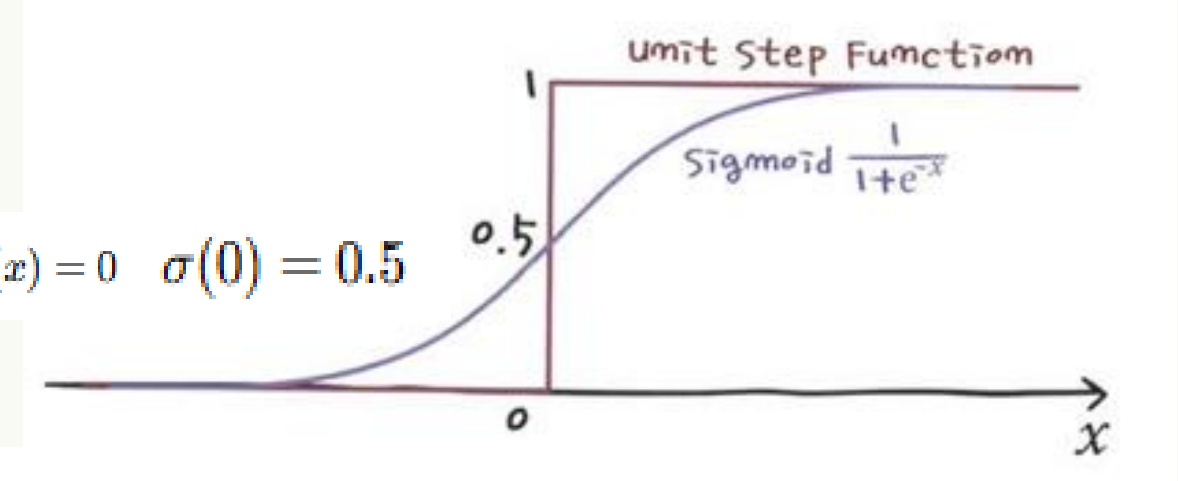
# 01 | Sigmoid

## Sigmoid

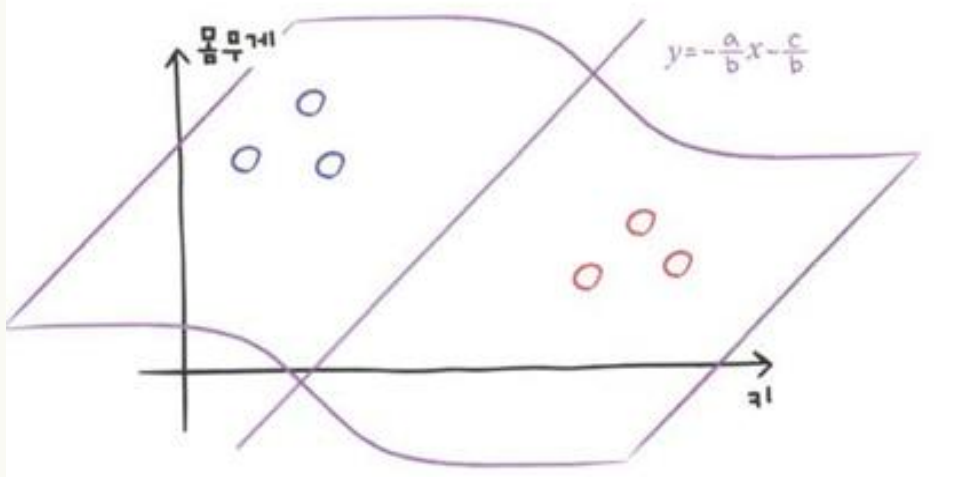
- Unit step function을 부드럽게 만들어보자

**Sigmoid** $\sigma(x) = \frac{1}{1+e^{-x}}$

- 1. 전구간 미분 가능 (gradient descent 가능)
- 2. 좀 더 부드러운 분류가 가능! $\lim_{x\to\infty}\sigma(x)=1$ $\lim_{x\to-\infty}\sigma(x)=0$ $\sigma(0)=0.5$

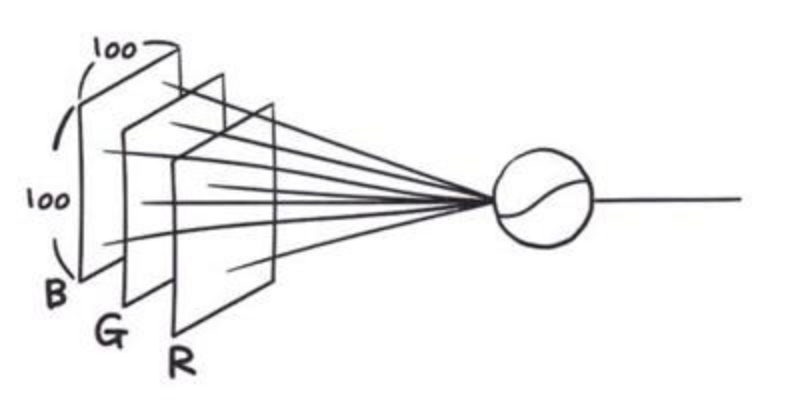



- 확률(혹은 정도)로 해석 가능
- 합리적인(가장 멀리 찢기) 분류 경계 선



# 01 | Sigmoid 이진 분류

- 입력을 3x100x100인 (Dog or Cat)사진이라고 하고 곧바로 출력층과 연결한다고 하자
  - 총 파라미터 개수? -> 이것이 곧 gradient의 길이 -> 이제 Loss 만 잘 정의하면 된다!




총 파라미터 개수 : 3*100*100 + 1 = 30001 -> 그라디언트 갯수

$$\begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ b \end{bmatrix}_{k+1} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ b \end{bmatrix}_{k} - \alpha \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \vdots \\ \frac{\partial L}{\partial b} \end{bmatrix} \quad 30{,}001$$

- 신경망이 사진을 입력 받아 **강아지일 확률을 출력하게끔** 만들어 봅시다 how?
- 강아지면 1, 고양이면 0에 나오도록 loss를 잘 정의하면 됨~!
  - 그냥 강아지 사진을 넣었을 때 0에 가까운 값이 나올수록 더 크게 혼내면 된다!
  - Loss로 "출력의 의미"를 컨트롤 한다!

# 01 | Sigmoid 이진 분류

- 신경망이 사진을 입력 받아 **강아지일 확률을 출력하게끔** 만들어 봅시다 how?
  - 그냥 강아지 사진을 넣었을 때 0에 가까운 값이 나올수록 더 크게 혼내면 된다!
  - Loss로 "출력의 의미"를 컨트롤 한다!
- 강아지면 1, 고양이면 0에 나오도록 loss를 잘 정의하면 됨~!
  - 여기에선 신경망 출력을 q, 정답을 y라고 지정 (강아지는 y = 1, 고양이는 y = 0)
  - "강아지 사진 입력" = "q = 1 나오게 하자" = "q를 키우자" --> sigmoid(0~1)
  - "고양이 사진 입력" = "q = 0 나오게 하자" = "1-q를 키우자" --> sigmoid(0~1)

If 강아지 사진
    maximize q
Else if 고양이 사진
    minimize q --> maximize –q --> maximize 1-q(확률로 해석 가능)

$$q^y(1-q)^{1-y}$$

# 01 | Sigmoid

## 이진 분류

$q^y(1-q)^{1-y}$ AI가 예측한 해당 동물일 확률

만약 Mini-Batch 방식으로 학습이 진행할 때는 여러 데이터에 대한 예측 확률을 곱함. (독립 시행)

$$q_1^{y_1}(1-q_1)^{1-y_1} \cdot q_2^{y_2}(1-q_2)^{1-y_2} \cdots q_{32}^{y_{32}}(1-q_{32})^{1-y_{32}}$$

곱한 확률(0~1)을 키워야 함 --> underflow 발생

underflow : 컴퓨터가 표현할 수 있는 가장 작은 양수보다 작은 수를 계산하려고 할 때 발생하는 현상, 결괏값이 0으로 처리 되어 정보 손실 발생

- log를 취하자 (취해도 되는 이유는?) --> 곱이 +로!

$$\log q_1^{y_1}(1-q_1)^{1-y_1} \cdot q_2^{y_2}(1-q_2)^{1-y_2} \cdots q_{32}^{y_{32}}(1-q_{32})^{1-y_{32}}$$

$$= \log q_1^{y_1}(1-q_1)^{1-y_1} + \log q_2^{y_2}(1-q_2)^{1-y_2} + \cdots + \log q_{32}^{y_{32}}(1-q_{32})^{1-y_{32}}$$

$$= \sum_{n=1}^{32} \log q_n^{y_n}(1-q_n)^{1-y_n}$$

- 0과 1 사이의 확률값을 -무한대 부터 0사이의 더 넓은 범위 값으로 변환

# 01 | Sigmoid

## 이진 분류

$$\log q_1^{y_1}(1-q_1)^{1-y_1} \cdot q_2^{y_2}(1-q_2)^{1-y_2} \cdots q_{32}^{y_{32}}(1-q_{32})^{1-y_{32}}$$
$$= \log q_1^{y_1}(1-q_1)^{1-y_1} + \log q_2^{y_2}(1-q_2)^{1-y_2} + \cdots + \log q_{32}^{y_{32}}(1-q_{32})^{1-y_{32}}$$
$$= \sum_{n=1}^{32} \log q_n^{y_n}(1-q_n)^{1-y_n}$$

$$L = \frac{1}{32}\sum_{n=1}^{32} -\log q_n^{y_n}(1-q_n)^{1-y_n}$$

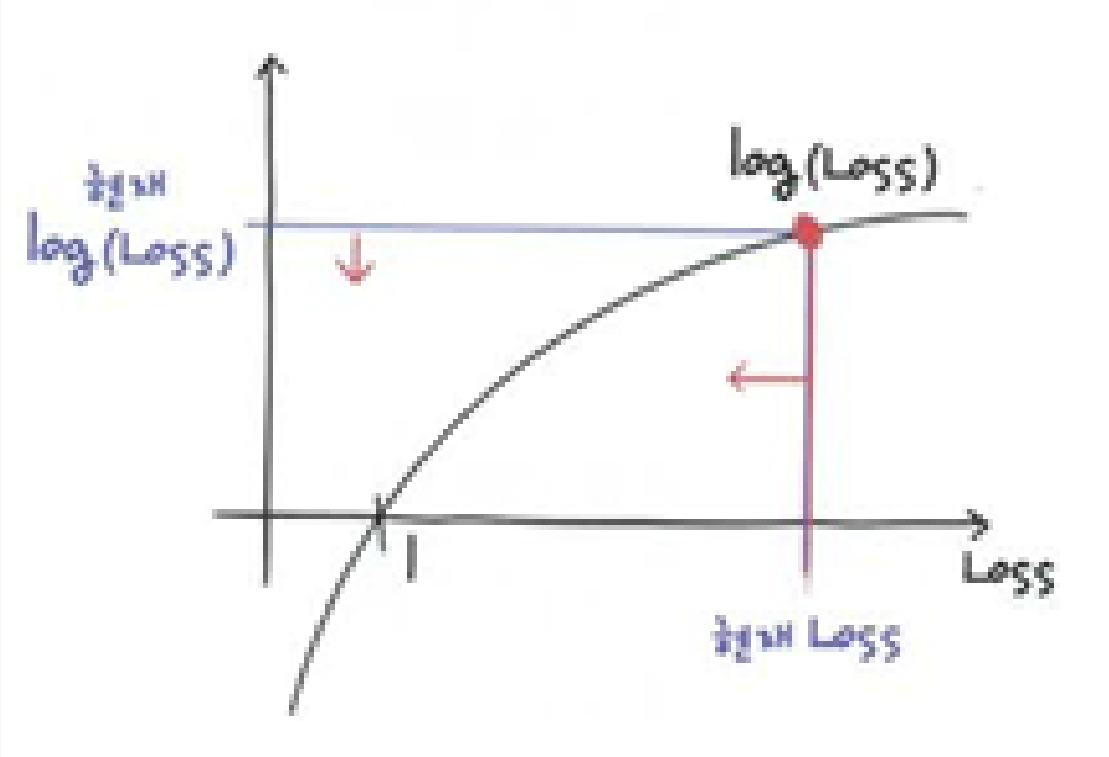
- **BCE Loss = 이진 분류에서 모델의 예측이 정답과 얼마나 다른지 수치화 하는 손실함수**

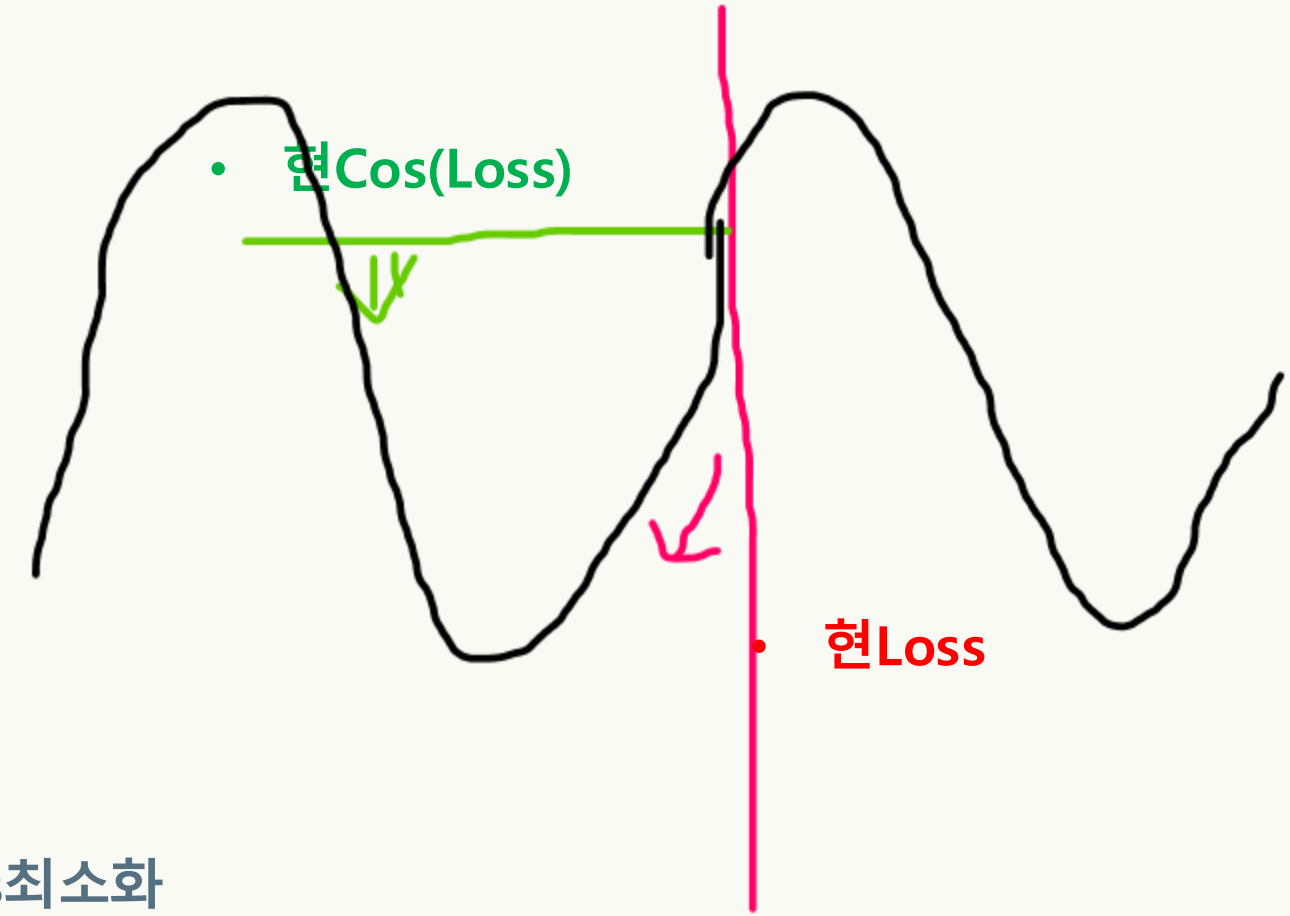
$$\begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ b \end{bmatrix}_{k+1} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ b \end{bmatrix}_{k} - \alpha \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \vdots \\ \frac{\partial L}{\partial b} \end{bmatrix} \quad 30{,}001$$

# 01 | Sigmoid

## 단조 증가 함수

- 단조 증가 함수




- Log를 취한 다음에 log(loss) 최소화 == loss최소화

# 01 | Logistic Regression

## 한계

- 앞에서 본 예제 -> 입력과 출력 사이의 관계를 확률 함수로 표현 -> 은닉층이 없는 인공 신경망으로 놓고 추정하는 방식 -> 로지스틱 회귀

- 로지스틱 회귀 -> 로지스틱 함수는 시그모이드 함수의 일반화 형태 -> 이름 유래
- 입력과 출력 사이의 관계를 logistic 함수 형태로 놓고, 그 함수의 파라미터를 추정
- 분류 문제를 선형식으로 점수화한 뒤, 그 점수를 확률로 변환해서 푼다.

# 01 | Logistic Regression 한계

**Level 2 : logistic regression은 logit을 linear regression으로 구한 것으로도 해석 가능 (Loss값을 얻기 위해 sigmoid를 통과 시켜 확률로 바꾼 것이다.)**

**Logit = Log(odds)**

**odds(승산) = q(승리 확률) / 1-q (패배 확률)**

**Logit => L = Log (q/1-q)**

$$e^L = \frac{q}{1-q}$$

$$e^L(1-q) = q$$

$$e^L - e^L q = q$$

$$e^L = q + e^L q$$

$$e^L = q(1+e^L)$$

$$q = \frac{e^L}{1+e^L}$$

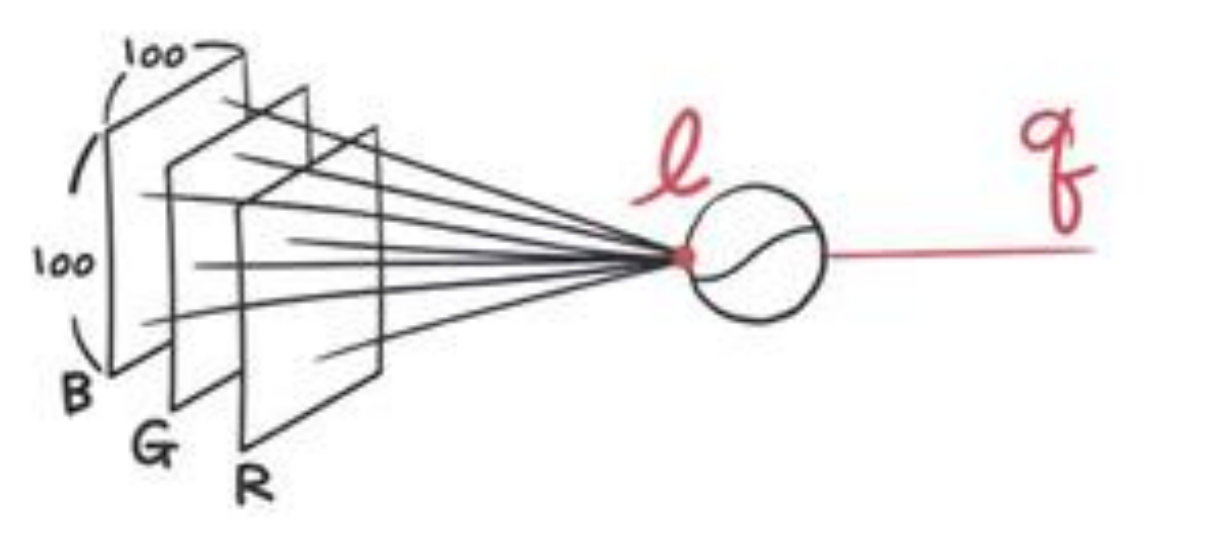
$$q = \frac{1}{1+e^{-L}}$$

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

**Sigmoid**

# 01 | Logistic Regression

## 한계

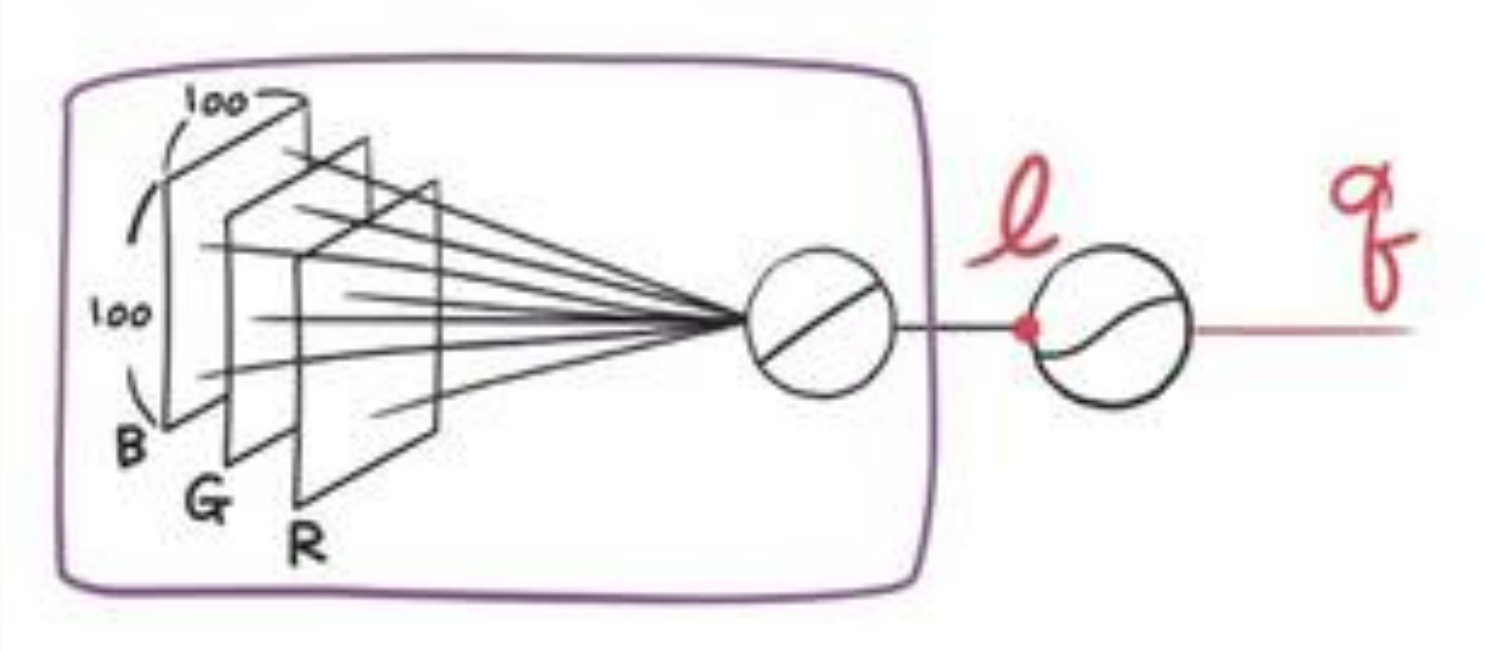



$$q = \frac{1}{1 + e^{-L}}$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

# 01 | Logistic Regression

## 한계




1. 사진(입력)과 Logit(출력)의 값이 linear하다.
2. Sigmoid는 BCE를 구하기 위해서는 확률로 바꿨어야했고 확률로 바꾸기 위해서 sigmoid를 사용했다.

# Unit Step Function, Sigmoid, BCE, Logistic Regression

감사합니다